

Research Progress Report

Project:	IsoShard Proxy: A Distributed CP-Mode Transaction Coordinator & SQL Optimizer on AWS
Proponent:	Sarthak Mushrif
Reporting Period:	Weeks 11, 12 & 13
Hours Logged:	20 Hours

1. Executive Summary of Progress

During this extended reporting period, development focused on resolving critical fault-tolerance vulnerabilities and preparing the middleware for cloud deployment. The primary achievement was the engineering of the Crash Recovery Daemon, which leverages the previously integrated AWS DynamoDB Write-Ahead Log (WAL) to resolve "phantom locks" left by proxy failures. Additionally, the compiler's Abstract Syntax Tree (AST) visitor was upgraded with "Predicate Pushdown" logic to drastically optimize SELECT query routing. Finally, the entire application was containerized using Docker, establishing a reproducible runtime environment.

2. Detailed Task Breakdown (20 Hours)

- **Crash Recovery Daemon Implementation (8 Hours):** Engineered a background reconciliation thread that executes upon proxy startup. The daemon queries AWS DynamoDB for any transactions in a PREPARED state that lack a terminal COMMITTED or ABORTED flag. It then issues compensating ROLLBACK commands to the involved PostgreSQL shards, ensuring database locks are safely released if the proxy process had previously crashed mid-transaction.
- **Predicate Pushdown Optimization (6 Hours):** Upgraded the IsoShardQueryVisitor to mathematically analyze SELECT statements. Instead of broadcasting a read query to all cluster nodes, the optimizer now detects targeted WHERE clauses (e.g., WHERE user_id = 100), hashes the value, and pushes the query down *only* to the single responsible physical shard, saving massive network bandwidth.
- **Docker Containerization (4 Hours):** Authored a multi-stage Dockerfile for the IsoShard application. The build stage compiles the Maven project and ANTLR4 generated classes, while the runtime stage utilizes a lightweight Amazon Corretto (Java 17) alpine image. Configured port exposure for the Netty TCP event loop.
- **Idempotency Testing (2 Hours):** Conducted simulated `kill -9` process terminations during Phase 1 of the 2PC protocol to verify the daemon's recovery logic.

3. Challenges & Roadblocks

- **Idempotent Rollbacks:** During recovery testing, issuing a ROLLBACK to a shard that had already auto-aborted due to a TCP timeout threw SQL exceptions. This was resolved by wrapping the shard-level recovery commands in TRY...CATCH blocks that ignore "transaction not found" errors, ensuring the daemon completes its reconciliation loop without crashing.
- **Docker Networking Constraints:** Moving the proxy into a Docker container broke the local HikariCP connections to the simulated database shards. This required reconfiguring the environment variables within the docker-compose.yml file to utilize host.docker.internal bridge networking for local validation before the upcoming AWS migration.

4. Objectives for Next Reporting Period (Weeks 14 & 15)

- Deploy the Dockerized IsoShard Proxy to **AWS EC2** instances managed by an Auto Scaling Group (ASG).
- Configure an **AWS Network Load Balancer (NLB)** to efficiently distribute incoming binary TCP traffic across multiple proxy instances.
- Begin researching and drafting the programmatic logic for distributed JOIN operations (combining data queried from two distinct shards in proxy memory).

Appendix A: Technical Artifact - Crash Recovery Daemon

The following Java snippet demonstrates the reconciliation logic executed upon proxy boot. It reads the DynamoDB WAL to resolve any hanging database locks caused by prior system crashes.

```
1 import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
2 import software.amazon.awssdk.services.dynamodb.model.*;
3 import java.util.Map;
4
5 public class CrashRecoveryDaemon {
6
7     private final DynamoDbClient dynamoClient;
8     private final String tableName = "IsoShard_WAL";
9
10    public CrashRecoveryDaemon() {
11        this.dynamoClient = DynamoDbClient.create();
12    }
13
14    /**
15     * Scans the WAL for transactions that were PREPARED but never finalized.
16     * This runs synchronously on proxy startup before Netty accepts traffic.
17     */
18    public void reconcileOrphanedTransactions(ShardManager shardManager) {
19        System.out.println("[Recovery] Scanning WAL for orphaned transactions
20        ...");
21
22        ScanRequest scanRequest = ScanRequest.builder()
23            .tableName(tableName)
24            .filterExpression("#s = :val")
25            .expressionAttributeNames(Map.of("#s", "State"))
26            .expressionAttributeValues(Map.of(":val", AttributeValue.builder().
27            s("PREPARED").build()))
28            .build();
29
30        ScanResponse response = dynamoClient.scan(scanRequest);
31
32        for (Map<String, AttributeValue> item : response.items()) {
33            String txId = item.get("TransactionId").s();
34            System.out.println("[Recovery] Found orphaned TxID: " + txId + ".
35            Issuing ROLLBACK.");
36
37            // Broadcast rollback to all shards. Idempotency ensures safety
38            // if a shard has already dropped the transaction.
39            shardManager.broadcastRollback(txId);
40
41            // Mark as safely aborted in the WAL
42            markAsAbortedInWal(txId);
43        }
44        System.out.println("[Recovery] Reconciliation complete.");
45    }
46
47    private void markAsAbortedInWal(String txId) {
48        // DynamoDB update logic to change state from PREPARED -> ABORTED
49    }
50 }
```

Listing 1: CrashRecoveryDaemon.java